

PES University

Department of Computer Science and Engineering

Lab 2

Agile Backlog Creation and Sprint Simulation in Jira

Problem Statement #12

Blood Bank Inventory and Emergency Donor Matcher

Satvik Kalyanpur

PES1UG24CS617

Section K

Jira project: Blood Bank Emergency Matcher (BBEM)

1. Overview

The five functional requirements and two non-functional requirements from Lab 1 were converted into a Jira backlog. I created a company-managed Scrum project called Blood Bank Emergency Matcher (key BBEM) and broke the requirements into six Epics containing sixteen User Stories in total, worth 91 story points.

Each requirement maps to exactly one Epic. The only requirement that does not get an Epic of its own is NFR-001, the rule that a blood bag must never be allocated to two requests at once. I kept that inside the request Epic as story BBEM-10, because the atomic locking it describes is a guarantee on the reservation step rather than a separate piece of work.

Every story is written in the As a / I want / So that format and carries a priority and a Fibonacci story point estimate. The work was split across two one-week sprints.

2. Epics

Key	Epic	Requirement	Stories	Points
BBEM-1	Emergency Blood Request and Fulfilment	FR-005, NFR-001	4	31
BBEM-2	Emergency Donor Matching and Alerts	FR-001	3	16
BBEM-3	Blood Inventory Management	FR-002	2	8
BBEM-4	Shelf Life and Expiry Governance	FR-003	3	13
BBEM-5	Donor Registration and Eligibility	FR-004	2	10
BBEM-6	Data Security, Privacy and Audit	NFR-002	2	13

Sprint 1 took the eight stories on the critical path, the request and fulfilment flow together with inventory registration and the expiry rules, which came to 44 points. Sprint 2 took the remaining eight, covering donor matching, donor registration and the security work, which came to 39 points. The ordering was deliberate: donors cannot be matched to a request before requests and inventory exist.

3. Screenshots

3.1 Backlog with Epics and User Stories

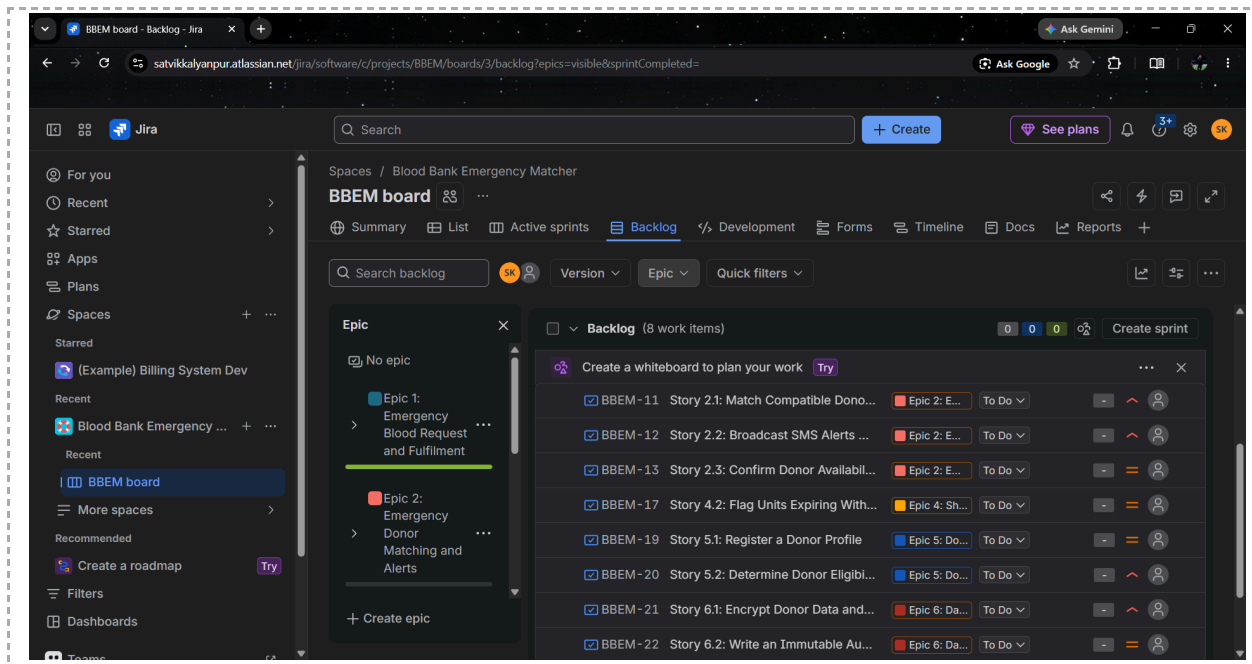


Figure 1: BBEM backlog showing the six Epics and their User Stories

3.2 Sprint Board and Story Points

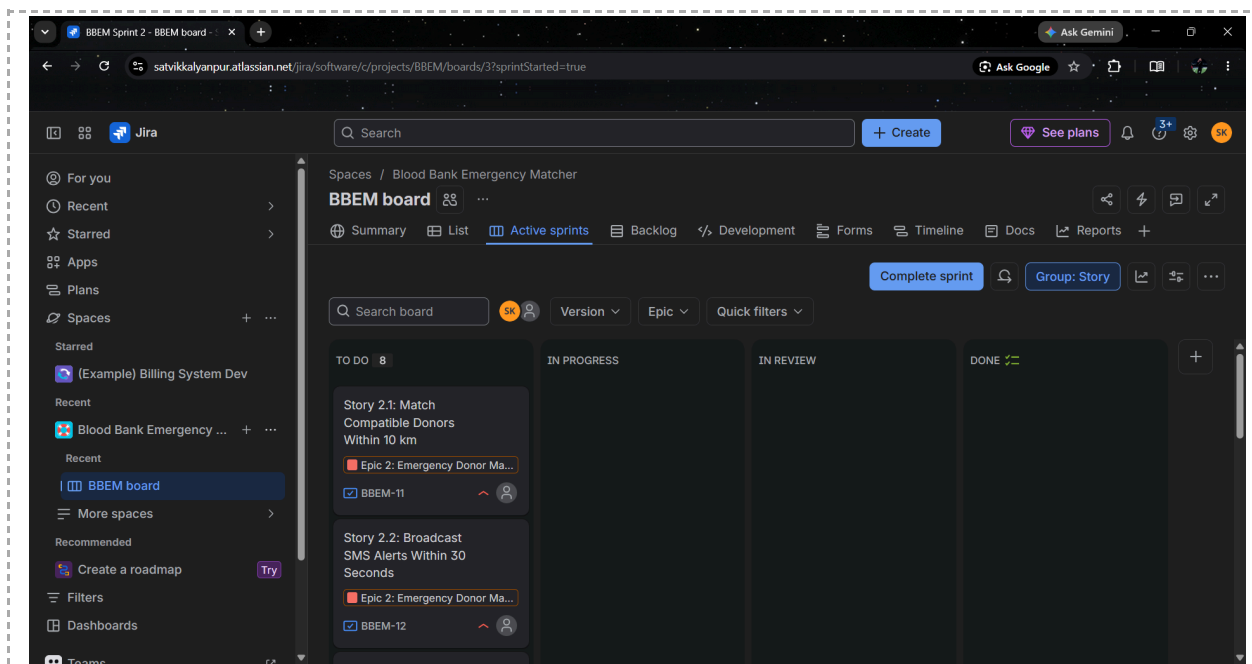


Figure 2: Active sprint board with story point estimates

3.3 Burndown Chart

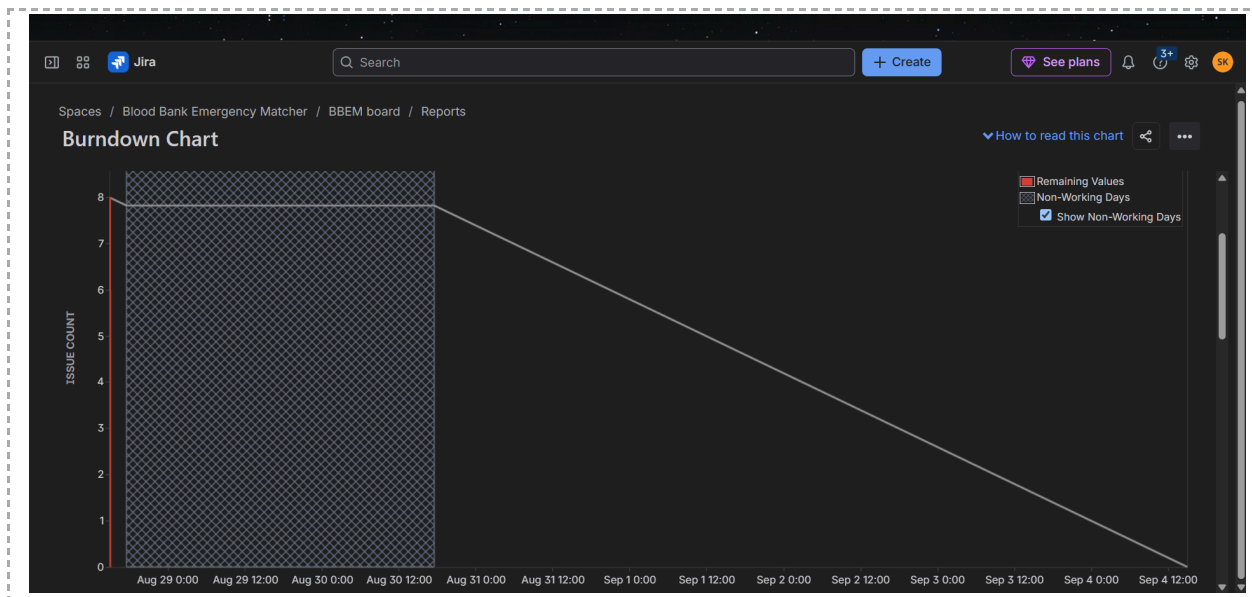


Figure 3: Sprint burndown chart

4. Reflection

Did your estimations reflect the actual effort?

Only loosely, and that is mostly because this was a simulation with no real implementation behind it. What I can say is that the relative sizing held up when I looked back at it. Giving 13 points to the story about preventing double allocation felt right next to the 3 points for validating that a bag ID is unique, since one is a concurrency problem with rollback and the other is a form check.

The estimate I am least confident about is the 8 points on encrypting donor data and setting up role-based access. That story bundles encryption at rest, encryption in transit and a permissions model into one item. If I were doing this again I would split it into two or three stories, because anything that large is really an admission that the team has not thought it through yet.

Was your backlog well-prioritized?

Broadly yes. I marked the two safety-critical stories as Highest, the one that stops a bag being promised to two patients and the one that quarantines expired units, since either failure could put a patient at risk. Everything that the emergency request flow depends on sat above the features that build on it.

One thing I would change is the position of donor eligibility. I gave it High priority but placed it in Sprint 2, even though the donor alert stories in that same sprint depend on eligibility being known. It works out because they are in the same sprint, but strictly the dependency should have pulled it earlier.

How did your simulated sprint align with your plan?

Not very closely, and the gap is worth being honest about. I planned two sprints of roughly equal size and that part held, 44 points then 39. But because this was a simulation I moved the stories from To Do

to In Progress to Done over the space of a few minutes rather than across a week. So the sprint finished exactly as planned, which in a real project would be suspicious rather than reassuring.

The useful lesson was less about the plan and more about the tooling. My story points were written into a Story Points field that the board was not configured to read, so the backlog reported zero points until I checked the board settings and found the estimation field it was actually using. Nothing in the sprint would have burned down correctly until that was fixed.

What insights did the burndown chart give about your team capacity?

Less than it should have, for the reason above. Because the stories were completed in one burst instead of over the week, the remaining line drops as a cliff rather than a slope, so there is no velocity trend to read from it. A chart like that tells you nothing about how much work the team can absorb.

What it did teach me is what makes a burndown useful in the first place. The chart is built from status changes as they happen, so it is only as honest as the team is about moving cards when the work actually moves. If everyone updates their tickets on the last day of the sprint, the chart will show a flat line followed by a drop and the team will have no early warning that they are behind. That is the real argument for updating the board daily.